

H8 单元程序设计报告

摘要

本文围绕 STAPpp 程序中 H8 单元的补充实现展开，目标是在保留原有总体框架、天空线存储和 LDLT 求解流程的前提下，将 8 节点六面体实体单元接入现有有限元主程序，并完成相应的验证与后处理支持。文中首先较系统地推导 H8 单元的等参映射、形函数、应变位移关系、本构矩阵以及单元刚度矩阵表达式；随后结合 STAPpp 既有的数据结构说明 H8 单元在材料类、单元组、输出模块和主程序入口中的接入方式；最后利用畸变网格分片试验、制造解收敛分析以及单 H8 块体算例对程序正确性进行验证。结果表明，所实现的 H8 单元能够通过 patch test，在网格加密下表现出合理的收敛趋势，并可在 C++ 主程序中完成完整的读入、组装、求解与应力输出。上述结果说明，当前程序已经形成了 H8 单元在 STAPpp 框架中的基本可用实现，并具备继续扩展后处理功能的基础。

关键词： H8 单元；六面体实体单元；STAPpp；有限元；分片试验；收敛性分析

1 课题背景与任务说明

本次课程大作业要求在原始 STAPpp 程序基础上扩展新的有限元单元，并完成相应的程序实现、分片试验、收敛性分析、验证算例以及前后处理支持。结合实体结构分析中对三维连续体离散的需求，本文选择 8 节点线性六面体实体单元 H8 作为扩展对象。H8 单元具有节点布置规整、几何表达直接、与工程软件常用实体单元形式一致等特点，是有限元入门与工程实现中最常见的一类三维实体单元。

从功能实现角度看，当前程序已经完成以下几个方面的工作：其一，在原始 C++ STAPpp 框架中补充了 H8 单元的数据读入、材料定义、单元刚度计算、总体组装、线性方程求解和单元平均应力输出，从而打通了 H8 单元在原程序中的完整计算流程；其二，保留并扩展了独立 Python 原型程序，用于自动生成验证网格、执行分片试验、开展制造解收敛分析，并输出 VTK 与文本结果文件；其三，围绕程序正确性给出了畸变网格 patch test、结构化网格收敛性分析以及 C++ 主程序单元算例验证，使程序不仅能够“运行起来”，而且能够以可量化方式说明结果是可信的。换言之，本文所实现的并非单一的 H8 刚度矩阵子程序，而是一套涵盖输入、求解、验证与基本后处理的 H8 单元扩展方案。

对原始仓库进行梳理后可以发现，C++ 主程序虽然在单元类型枚举中预留了 H8 名称，但程序真正实现的只有三维杆单元 Bar；与 H8 相关的数据读入、单元刚度、材料参数、应力恢复和结果输出均不存在。因此，本次工作的实质并不是对现有 H8 模块进行修补，而是要在不破坏原有 STAPpp 主体框架的前提下，将 H8 单元从“类型占位”补充为“可实际计算”的实体单元。

围绕这一目标，本文采取了两条并行但互补的实现路线。一方面，在原始 C++ STAPpp 框架中接入 H8 单元，使主程序能够完成 H8 单元的输入、组装、求解和输出；另一方面，保留一份独立的 Python 原型程序 STAPpy_h8.py，用于快速实施分片试验、收敛分析以及 ParaView 后处理导出。前者用于满足“在原程序框架内实现新单元”的核心要求，后者则提高了验证与展示环节的效率。基于这两条路线，本文将从单元理论、程序结构、数值验证和结果分析几个方面对整个实现过程进行说明。

2 H8 单元理论基础

2.1 单元自由度与插值

H8 单元为 8 节点线性六面体实体单元，每个节点包含 x 、 y 、 z 三个方向的平移自由度，因此单元位移参数总数为

$$N_D = 8 \times 3 = 24. \quad (1)$$

为了统一处理规则几何与畸变几何，H8 单元采用等参思想进行描述。即在母单元空间中引入自然坐标 (ξ, η, ζ) ，其取值范围均为 $[-1, 1]$ ，并在该坐标系内定义节点位置、几何映射和位移插值函数。8 个节点在母单元中的局部坐标分别为

$$\begin{aligned} &(-1, -1, -1), (1, -1, -1), (1, 1, -1), (-1, 1, -1), \\ &(-1, -1, 1), (1, -1, 1), (1, 1, 1), (-1, 1, 1). \end{aligned} \quad (2)$$

由三维张量积插值可得 H8 单元第 i 个节点的形函数

$$N_i(\xi, \eta, \zeta) = \frac{1}{8}(1 + \xi\xi_i)(1 + \eta\eta_i)(1 + \zeta\zeta_i). \quad (3)$$

其中 (ξ_i, η_i, ζ_i) 表示第 i 个节点在母单元中的坐标。上述形函数满足 Kronecker 乘积性质和分片统一条件，即 $N_i(\xi_j, \eta_j, \zeta_j) = \delta_{ij}$ ，并且

$$\sum_{i=1}^8 N_i(\xi, \eta, \zeta) = 1. \quad (4)$$

因此，单元内任一点的几何坐标与位移场均可采用相同的形函数展开：

$$\mathbf{x}(\xi, \eta, \zeta) = \sum_{i=1}^8 N_i(\xi, \eta, \zeta) \mathbf{x}_i, \quad (5)$$

$$\mathbf{u}(\xi, \eta, \zeta) = \sum_{i=1}^8 N_i(\xi, \eta, \zeta) \mathbf{u}_i. \quad (6)$$

记单元节点位移列阵为

$$\mathbf{d}_e = [u_1 \ v_1 \ w_1 \ \cdots \ u_8 \ v_8 \ w_8]^T, \quad (7)$$

则位移插值可以进一步写成矩阵形式

$$\mathbf{u} = N \mathbf{d}_e, \quad (8)$$

其中 N 为 3×24 的块对角型形函数矩阵。这一写法为后续推导应变矩阵和刚度矩阵提供了统一表达。

需要特别说明的是，H8 单元虽然在母单元空间中对应一个规则立方体，但这并不意味着其在物理空间中只能表示长方体或正交块体。H8 的核心在于“8 节点线性六面体等参映射”，而不在于物理几何必须保持规则。只要给定 8 个节点在物理空间中的坐标，程序就可以利用相同的形函数把母单元映射到实际单元，因此物理空间中的 H8 单元既可以是规则长方体，也可以是一般的凸六面体，甚至可以带有一定程度的边面倾斜和几何畸变。

从等参映射角度看，单元几何是否规则由节点坐标决定，而不是由单元类型名称决定。若 8 个节点恰好落在规则块体的 8 个顶点上，则映射后的物理单元就是长方体；若节点坐标发生剪切、拉伸或扭曲，则映射后的物理单元就成为一般六面体。因此，H8 单元的适用对象应理解为“可由线性六面体映射描述的实体单元”，而不是“只能处理长方体网格的单元”。

当然，H8 单元对几何形状也并非没有约束。为了使映射有效并保证数值积分正确，通常要求单元在高斯积分点处满足 $\det(J) > 0$ ，即母单元到物理单元的映射不能发生翻转；同时，单元的畸变程度也不宜过大，否则会造成应变矩阵恶化、刚度矩阵条件数变差以及计算精度下降。换言之，H8 允许一般六面体，但并不允许任意失真的“坏单元”。

本次工作中之所以在不同部分出现了“规则块体”和“畸变六面体”两类几何，正是出于不同验证目的的考虑。理论上，为证明程序并不依赖长方体假设，Python 分片试验采用了畸变 $2 \times 2 \times 2$ H8 网格；实践上，为了在 C++ 主程序中快速打通最小算例闭环并降低输入文件组织难度，示例验证算例则优先选用了规则块体几何。因此，当前报告和代码中的块体算例只是验证策略上的选择，而不是 H8 单元本身的限制。

2.2 应变矩阵与本构矩阵

在线弹性小变形条件下，三维应变采用工程应变表示为

$$\boldsymbol{\varepsilon} = [\varepsilon_{xx} \ \varepsilon_{yy} \ \varepsilon_{zz} \ \gamma_{xy} \ \gamma_{yz} \ \gamma_{xz}]^T. \quad (9)$$

根据位移梯度定义，正常应变和工程剪应变分别满足

$$\varepsilon_{xx} = \frac{\partial u}{\partial x}, \quad \varepsilon_{yy} = \frac{\partial v}{\partial y}, \quad \varepsilon_{zz} = \frac{\partial w}{\partial z}, \quad (10)$$

$$\gamma_{xy} = \frac{\partial u}{\partial y} + \frac{\partial v}{\partial x}, \quad \gamma_{yz} = \frac{\partial v}{\partial z} + \frac{\partial w}{\partial y}, \quad \gamma_{xz} = \frac{\partial u}{\partial z} + \frac{\partial w}{\partial x}. \quad (11)$$

将等参位移插值代入上式，并把节点自由度收集为单元位移列阵后，可写成经典的应变位移关系

$$\boldsymbol{\varepsilon} = B \mathbf{d}_e. \quad (12)$$

其中 B 为 6×24 的应变矩阵。若记第 i 个节点形函数对物理坐标的偏导数为

$$N_{i,x} = \frac{\partial N_i}{\partial x}, \quad N_{i,y} = \frac{\partial N_i}{\partial y}, \quad N_{i,z} = \frac{\partial N_i}{\partial z}, \quad (13)$$

则第 i 个节点对应的应变矩阵子块可写成

$$B_i = \begin{bmatrix} N_{i,x} & 0 & 0 \\ 0 & N_{i,y} & 0 \\ 0 & 0 & N_{i,z} \\ N_{i,y} & N_{i,x} & 0 \\ 0 & N_{i,z} & N_{i,y} \\ N_{i,z} & 0 & N_{i,x} \end{bmatrix}, \quad (14)$$

从而有

$$B = [B_1 \ B_2 \ \cdots \ B_8]. \quad (15)$$

对于三维各向同性线弹性材料，本构关系满足广义胡克定律

$$\boldsymbol{\sigma} = D \boldsymbol{\varepsilon}. \quad (16)$$

式中应力向量写成

$$\boldsymbol{\sigma} = [\sigma_{xx} \ \sigma_{yy} \ \sigma_{zz} \ \tau_{xy} \ \tau_{yz} \ \tau_{xz}]^T. \quad (17)$$

若以弹性模量 E 和泊松比 ν 作为材料参数，则可先引入拉梅常数

$$\lambda = \frac{E\nu}{(1+\nu)(1-2\nu)}, \quad \mu = \frac{E}{2(1+\nu)}. \quad (18)$$

于是三维本构矩阵 D 可写为

$$D = \begin{bmatrix} \lambda + 2\mu & \lambda & \lambda & 0 & 0 & 0 \\ \lambda & \lambda + 2\mu & \lambda & 0 & 0 & 0 \\ \lambda & \lambda & \lambda + 2\mu & 0 & 0 & 0 \\ 0 & 0 & 0 & \mu & 0 & 0 \\ 0 & 0 & 0 & 0 & \mu & 0 \\ 0 & 0 & 0 & 0 & 0 & \mu \end{bmatrix}. \quad (19)$$

当 $E > 0$ 且 $-1 < \nu < 0.5$ 时, 材料矩阵保持正定, 单元刚度矩阵也由此具备合理的数值性质。在程序实现中, H8 材料类实际保存的正是 E 与 ν 两个参数, 单元层面则通过它们即时构造矩阵 D 。

2.3 雅可比矩阵与数值积分

由于形函数最初定义在自然坐标系中, 而应变矩阵需要形函数对物理坐标 (x, y, z) 的导数, 因此必须借助坐标映射完成变量转换。由等参几何映射可得

$$x = \sum_{i=1}^8 N_i x_i, \quad y = \sum_{i=1}^8 N_i y_i, \quad z = \sum_{i=1}^8 N_i z_i. \quad (20)$$

进一步写成微分关系, 可引入雅可比矩阵

$$J = \frac{\partial(x, y, z)}{\partial(\xi, \eta, \zeta)}. \quad (21)$$

展开后有

$$J = \begin{bmatrix} \frac{\partial x}{\partial \xi} & \frac{\partial y}{\partial \xi} & \frac{\partial z}{\partial \xi} \\ \frac{\partial x}{\partial \eta} & \frac{\partial y}{\partial \eta} & \frac{\partial z}{\partial \eta} \\ \frac{\partial x}{\partial \zeta} & \frac{\partial y}{\partial \zeta} & \frac{\partial z}{\partial \zeta} \end{bmatrix}. \quad (22)$$

于是自然坐标导数与物理坐标导数之间满足

$$\begin{bmatrix} \frac{\partial N_i}{\partial x} \\ \frac{\partial N_i}{\partial y} \\ \frac{\partial N_i}{\partial z} \end{bmatrix} = J^{-1} \begin{bmatrix} \frac{\partial N_i}{\partial \xi} \\ \frac{\partial N_i}{\partial \eta} \\ \frac{\partial N_i}{\partial \zeta} \end{bmatrix}. \quad (23)$$

当 $\det(J) > 0$ 时, 说明母单元到物理单元的映射方向正确, 单元几何有效。基于虚功原理或最小势能原理, H8 单元的刚度矩阵可写为

$$K_e = \int_{V_e} B^T D B \, dV. \quad (24)$$

将积分区域变换到母单元空间后, 上式可写成

$$K_e = \int_{-1}^1 \int_{-1}^1 \int_{-1}^1 B^T D B \det(J) \, d\xi \, d\eta \, d\zeta. \quad (25)$$

由于 H8 为线性六面体单元, 本文采用 $2 \times 2 \times 2$ Gauss 全积分。8 个高斯点在各方向上的取值均为

$$\xi, \eta, \zeta = \pm \frac{1}{\sqrt{3}}. \quad (26)$$

各高斯点权重均为 1，因此刚度矩阵的数值积分表达为

$$K_e \approx \sum_{g=1}^8 B_g^T D B_g \det(J_g). \quad (27)$$

在程序实现中，每个高斯点依次完成自然导数计算、雅可比矩阵求逆、应变矩阵组装以及刚度子项累加。得到的 24×24 单元全矩阵再被压缩为 STAPpp 所需的一维上三角存储形式，最终参与天空线总体组装。

3 程序设计思路

3.1 总体思路

原始 STAPpp 主程序已经具备较成熟的总体有限元求解框架：CDomain 负责输入数据和全局对象组织，CElementGroup 负责单元与材料管理，CSkylineMatrix 负责天空线矩阵存储，CLDLTSolver 负责方程组分解与回代，COutputter 负责输入和结果输出。因此，本次工作并未重写总体求解流程，而是遵循既有架构，在“单元定义层”补充 H8 所需的最小功能闭环。

具体而言，程序首先新增 H8 单元类和 H8 材料类，使单元能够独立完成节点读入、刚度矩阵计算和应力恢复；随后在单元组分配逻辑中注册 H8 对应的对象大小、材料大小与分配分支，使 Domain 在读入 H8 单元组时能够正确创建对象；最后扩展输出模块，使主程序能够输出 H8 材料定义、连接信息以及单元平均应力。通过这种做法，原有 Domain $\rightarrow$ SkylineMatrix $\rightarrow$ LDLT $\rightarrow$ Outputter 主流程保持不变，而 H8 单元则作为新的局部实现嵌入到整体框架中。

3.2 C++ 主程序接入位置

H8 单元在 C++ STAPpp 主程序中的接入主要涉及以下几个文件：单元头文件 `src/h/H8.h`，单元实现文件 `src/cpp/H8.cpp`，材料扩展文件 `src/h/Material.h` 与 `src/cpp/Material.cpp`，单元组注册文件 `src/h/ElementGroup.h` 与 `src/cpp/ElementGroup.cpp`，以及输出扩展文件 `src/h/Outputter.h` 与 `src/cpp/Outputter.cpp`。此外，`src/cpp/main.cpp` 中也补充了对应头文件引用，以保证主程序在链接阶段能够识别 H8 单元实现。

从函数职责看，H8 单元类的核心接口仍然与 STAPpp 中已有单元保持一致。其中，Read 负责读取 8 个节点号与材料号，Write 负责将单元连接关系写入结果文件。ElementStiffness 用于形成 24×24 单元刚度矩阵，ElementStress 则根据求得的单元位移恢复应力并进行输出。这种接口一致性保证了 H8 单元可以无缝接入原有单元组和总体求解流程，而不必改写上层框架。

3.3 Python 原型的作用

虽然 H8 已接入 C++ 主程序，但为了更高效地完成课程作业中的分片试验、收敛分析和可视化展示，本文仍保留了 Python 原型 STAPpy_h8.py。该原型更适合快速构造标准 patch test 网格、直接计算高斯点应力误差、生成收敛分析 CSV 文件，并导出 ParaView 可读取的 VTK 文件。因此，C++ 主程序与 Python 原型在本文中并非相互替代关系，而是分别承担“接入原框架”和“高效验证展示”两类不同任务。

4 H8 单元在 C++ 中的实现细节

4.1 数据读入

H8 单元的 Read 函数在 src/cpp/H8.cpp 中实现。每个单元数据行格式为

$$\text{element } n_1 \ n_2 \ n_3 \ n_4 \ n_5 \ n_6 \ n_7 \ n_8 \ mset. \quad (28)$$

程序读入后，会将 8 个节点编号转换为对应的 CNode* 指针，并将材料集合号映射为 CH8Material*。这一做法延续了 STAPpp 原有的对象管理方式，使单元在后续组装阶段可以直接访问节点坐标、自由度编号和材料参数。

4.2 材料定义

H8 材料类 CH8Material 在 src/h/Material.h 和 src/cpp/Material.cpp 中定义。与杆单元不同，H8 单元不再依赖截面积等一维参数，而是直接采用三维各向同性线弹性材料最常用的两个基本参数，即弹性模量 E 与泊松比 ν 。材料输入格式为

$$nset \ E \ \nu. \quad (29)$$

在单元计算时，程序根据这两个参数现场构造三维本构矩阵 D ，从而避免在材料层存储冗余矩阵数据，也与原有 STAPpp 材料类的轻量化风格保持一致。

4.3 刚度矩阵计算

H8 单元的刚度矩阵形成过程严格遵循上一节给出的理论表达。程序在 8 个高斯点上首先计算形函数关于自然坐标的导数，再由节点坐标构造雅可比矩阵及其逆矩阵，将自然导数转换为物理导数；随后拼装 6×24 的应变矩阵 B ，并与材料矩阵 D 一起形成高斯点刚度贡献 $B^T D B \det(J)$ 。所有高斯点的贡献累加后，得到 24×24 的单元全刚矩阵。

考虑到 STAPpp 总体求解器使用天空线格式存储刚度矩阵，单元内部虽然先形成全矩阵，但在返回装配前仍需按照程序原有规则压缩为上三角一维存储格式。这样既保持了实现过程的清晰性，又保证了 H8 单元与原有总体矩阵装配接口兼容。

4.4 应力恢复

当前 H8 应力输出采用 8 个高斯点应力平均的方式。程序首先依据定位矩阵抽取单元 24 个自由度位移，再在每个高斯点上计算 $\varepsilon = Bd_e$ 与 $\sigma = D\varepsilon$ ，最后对 8 个高斯点应力做算术平均，并将所得结果作为该单元的文本输出值。虽然这种处理方式不如逐高斯点输出更为完整，但对于当前课程设计中的结果核对和程序验证而言已经足够。

5 输入数据格式设计

5.1 C++ 主程序输入格式

由于 STAPpp 主程序沿用 STAP90 风格的 .dat 文本输入，因此 H8 单元也遵循相同格式。本文给出的样例输入文件为 data/h8_block.dat，其整体结构依次包含标题行、控制行 NUMNP NUMEG NLCASE MODEX、节点数据、荷载工况数据、单元组控制行、材料数据以及单元连接数据。对于 H8 单元组，单元组控制行写为 4 NUME NUMMAT，其中单元类型号 4 在当前实现中被用作 H8；材料行写为 nset E nu；单元行则写为 element node1 node2 node3 node4 node5 node6 node7 node8 mset。这种设计的优点在于它最大程度继承了 STAPpp 原有输入习惯，从而避免因引入新单元而改动全局读入流程。

5.2 Python 原型输入格式

为了便于快速搭建验证算例，Python 原型采用 JSON 输入格式，样例见 data/h8_cantilever.json。该文件中包含标题、材料、节点、单元、位移边界、节点荷载、体力项和输出控制等字段。相较于 .dat 文本格式，JSON 更适合脚本自动构网、批量生成载荷与边界条件，并直接组织可视化输出路径，因此更适合作为 patch test 与收敛性分析阶段的输入介质。

6 前处理与后处理设计

6.1 前处理

前处理的内容主要包括节点坐标生成、单元连接关系确定、材料参数指定、边界条件施加以及节点荷载或体力项施加。在 C++ 主程序中，这些信息通过手工编写 .dat 文件给出；而在 Python 原型中，前处理既可通过 JSON 文件直接指定，也可通过脚本自动生成结构化网格，从而更方便地开展参数化验证。

6.2 后处理

后处理分为两个层面。其一是 C++ 主程序的文本输出，主要包含节点位移、材料信息和单元平均应力，对应结果文件为 data/h8_block.out；其二是 Python 原型的 VTK 导出，能够输出位移矢量、位移模以及 6 个应力分量，并可在 ParaView 中绘制变形图和应力云图，对应文件包括 outputs/h8_cantilever_summary.txt、outputs/h8_cantilever.vtk 和 outputs/h8_convergence.csv。前者用于验证 STAPpp 主程序中的接入效果，后者用于加强结果展示与误差分析。

7 使用方法

本文程序的使用分为 C++ 主程序和 Python 原型两条路径。对于 C++ 主程序，首先在 src 目录下使用 CMake 配置并编译 STAPpp 工程，生成可执行文件 stap++.exe；随后准备符合 STAP90 风格的 H8 输入文件，例如 data/h8_block.dat，并以该算例名作为输入运行主程序；程序完成求解后，将在同目录下生成文本结果文件 data/h8_block.out，其中可直接查看节点位移和单元平均应力。该路径主要用于说明 H8 单元已经真正接入原始 STAPpp 求解框架。

对于 Python 原型，可直接运行 STAPpy_h8.py 脚本执行不同功能。若需进行分片试验，可运行 patch-test 模式以输出中心节点位移误差、高斯点应力误差和自由自由度残差；若需进行收敛分析，可运行 convergence 模式自动生成不同网格密度下的误差数据，并将结果写入 outputs/h8_convergence.csv；若需对一般算例进行求解，则可读取 JSON 输入文件，例如 data/h8_cantilever.json，并输出文本摘要与 VTK 文件，随后在 ParaView 中查看位移和应力云图。两条路径分别服务于“原框架接入”和“数值验证与展示”这两个层面的需求。

8 H8 单元分片试验

8.1 试验目的

分片试验用于检验单元是否能够精确重现线性位移场以及对应的常应变、常应力状态。对于线性完全单元而言，若其通过 patch test，说明其基本插值、应变恢复和数值积分实现是正确的。

8.2 试验设置

本次分片试验在 Python 原型中完成，采用畸变 $2 \times 2 \times 2$ H8 网格，共 8 个单元、27 个节点，其中边界节点 26 个，内部自由节点 1 个。该网格并非规则块体，而是由结

构化参数网格经过线性映射得到。具体映射写为

$$\mathbf{x} = \mathbf{x}_0 + T \begin{bmatrix} \xi \\ \eta \\ \zeta \end{bmatrix}, \quad (30)$$

其中偏移向量与映射矩阵分别取为

$$\mathbf{x}_0 = \begin{bmatrix} -0.6 \\ -0.73 \\ -0.25 \end{bmatrix}, \quad T = \begin{bmatrix} 1.30 & 0.20 & 0.10 \\ 0.00 & 1.10 & 0.16 \\ 0.08 & 0.12 & 0.90 \end{bmatrix}. \quad (31)$$

因此，该试验能够有效检验程序在一般六面体映射下的几何变换和积分实现，而不是只在规则长方体条件下做验证。网格中心唯一内部自由节点的物理坐标为

$$(x, y, z) = (0.2, -0.1, 0.3). \quad (32)$$

在该网格上施加如下线性位移场：

$$u = 0.010x + 0.002y - 0.001z, \quad (33)$$

$$v = -0.003x + 0.015y + 0.004z, \quad (34)$$

$$w = 0.002x - 0.005y + 0.020z. \quad (35)$$

对应精确常应变为

$$\varepsilon = [0.010, 0.015, 0.020, -0.001, -0.001, 0.001]^T. \quad (36)$$

材料参数取为 $E = 1000.0$ 、 $\nu = 0.3$ 。由三维本构矩阵可进一步得到理论常应力为

$$\sigma = [33.65384615, 37.50000000, 41.34615385, -0.38461538, -0.38461538, 0.38461538]^T. \quad (37)$$

边界节点全部施加精确位移，只保留中心节点为自由节点。这样构造的意义在于：若单元和程序实现正确，则内部自由节点数值位移应与精确线性位移场完全一致，且 8 个单元全部 64 个高斯积分点处恢复得到的应力都应与上述理论常应力一致。换言之，该试验同时考察了形函数插值、雅可比矩阵变换、应变矩阵构造以及 $2 \times 2 \times 2$ Gauss 积分是否实现一致。

为便于展示，表 1 汇总了本次分片试验的主要设置。

8.3 分片试验结果

程序运行输出表明，中心自由节点的数值位移与理论位移完全一致。对应结果见表 2。

表 1: H8 分片试验设置

项目	取值
单元类型	8 节点线性六面体 H8
网格划分	$2 \times 2 \times 2$
单元数	8
节点数	27
边界节点数	26
内部自由节点数	1
材料参数	$E = 1000.0, \nu = 0.3$
积分格式	$2 \times 2 \times 2$ Gauss 全积分
内部自由节点坐标	$(0.2, -0.1, 0.3)$

表 2: 中心自由节点位移对比

量	u	v	w
理论位移	0.0015	-0.0009	0.0069
数值位移	0.0015	-0.0009	0.0069
绝对误差	1.084202×10^{-18} (最大分量误差)		

除位移外，程序还对所有高斯积分点应力进行检查。理论常应力与数值恢复应力之间的最大误差仅为 1.421085×10^{-14} ，自由自由度平衡残差最大值为 1.776357×10^{-15} ，而约束自由度反力最大值为 1.478625×10^1 。这里反力并不是误差量，而是边界位移约束为维持该线性场所自然产生的支反力，因此不要求其为零。将关键判据汇总如表 3 所示。

表 3: H8 分片试验关键结果

指标	数值
理论常应变	$[0.010, 0.015, 0.020, -0.001, -0.001, 0.001]^T$
理论常应力	$[33.65384615, 37.50000000, 41.34615385, -0.38461538, -0.38461538, 0.38461538]^T$
中心节点最大位移误差	1.084202×10^{-18}
高斯点最大应力误差	1.421085×10^{-14}
自由自由度最大残差	1.776357×10^{-15}
约束自由度最大反力	1.478625×10^1

从上述结果可以看出，位移误差、应力误差和自由自由度残差都处于机器舍入误差量级。这说明当前 H8 单元程序在畸变六面体网格下仍能够精确再现线性位移场、常应变场和常应力场，因而其形函数插值、坐标变换、应变恢复与数值积分实现彼此一致，分片试验可判定为通过。更重要的是，由于该试验不是在规则块体上完成，而是在一般畸变六面体上完成，因此它同时说明当前实现并不依赖“长方体单元”的特殊几何假设。

9 收敛性分析

9.1 分析思路

收敛性分析采用制造解方法，在 Python 原型中施加二次位移场：

$$u = 0.02x^2, \quad v = -0.015y^2, \quad w = 0.01z^2. \quad (38)$$

再根据解析应力散度构造一致体力项，在结构化网格下进行数值求解，并观察网格加密后的误差变化。

9.2 收敛数据

收敛数据见 `outputs/h8_convergence.csv`，整理如下。

每边单元数	单元数	节点数	位移误差	应力误差	残差
1	1	8	0.000000e+00	2.2886568241e+01	0.000000e+00
2	8	27	6.9388939039e-18	1.1443284120e+01	0.000000e+00
4	64	125	6.9388939039e-18	5.7216420602e+00	2.3314683517e-15

9.3 结果分析

由表中数据可以看出，随着网格从 $1 \times 1 \times 1$ 逐步加密到 $4 \times 4 \times 4$ ，高斯点应力误差呈现稳定下降趋势，整体上近似减半；与此同时，内部位移误差始终保持在极小量级，自由自由度残差也始终维持在舍入误差范围内。由此可见，当前 H8 单元程序不仅在分片试验中表现正确，而且在制造解驱动下也显示出合理的数值收敛性，能够满足课程作业对单元稳定性和精度的基本要求。

10 C++ 主程序验证算例

10.1 算例模型

为验证 H8 单元在原始 STAPpp 主程序中的实际可用性，本文构造了单 H8 块体算例 data/h8_block.dat。该模型的几何尺寸为 $1.0 \times 0.2 \times 0.2$ ，共包含 1 个 H8 单元和 8 个节点，材料参数取为 $E = 2.10 \times 10^{11}$ 、 $\nu = 0.30$ 。计算中将左端 4 个节点全部约束，在右端 4 个节点分别施加 $F_z = -250$ 的节点力，用以形成一个最小但完整的三维实体受载算例。

之所以在 C++ 主程序验证阶段采用规则块体而不是畸变六面体，主要出于三个考虑。首先，规则块体具有最清晰的节点拓扑关系和边界条件表达，便于在 STAP90 风格的 .dat 输入中直接书写和检查；其次，规则几何可以降低几何畸变对结果的附加影响，使这一算例更聚焦于验证“主程序是否已完成 H8 的读入、组装、求解和输出闭环”；最后，规则块体也是最简洁、最稳妥的基准模型，便于将注意力集中在程序实现本身而不是复杂几何因素上。因此，这一算例只能说明本文在 C++ 框架中首先选取了最简洁、最稳妥的基准模型，并不能推出 H8 单元只能用于长方体几何。

10.2 C++ 主程序运行结果

主程序成功编译并运行，生成输出文件 data/h8_block.out。自由端 4 个节点的位移结果整理如表 4 所示。

表 4: C++ 主程序 H8 验证算例自由端节点位移

节点号	U_x	U_y	U_z
2	-3.74771×10^{-10}	-2.76558×10^{-7}	-2.09465×10^{-7}
3	-1.33428×10^{-8}	9.24718×10^{-8}	-1.98267×10^{-8}
6	7.55974×10^{-8}	3.43702×10^{-8}	-9.05205×10^{-8}
7	-2.34551×10^{-7}	-1.65850×10^{-7}	-5.52194×10^{-8}

对应的单元平均应力为

$$\sigma_{xx} = 2.60149 \times 10^4, \quad (39)$$

$$\sigma_{yy} = 6.70745 \times 10^4, \quad (40)$$

$$\sigma_{zz} = 4.98592 \times 10^4, \quad (41)$$

$$\tau_{xy} = -3.89943 \times 10^4, \quad (42)$$

$$\tau_{yz} = 2.80214 \times 10^4, \quad (43)$$

$$\tau_{xz} = -2.22360 \times 10^4. \quad (44)$$

10.3 结果说明

为了更直观地展示该验证算例中的位移与应力分布，本文进一步将求解结果导入 ParaView 进行后处理，并分别绘制六个应力分量云图以及一个位移模云图。与前文表格中的节点位移和单元平均应力数据相比，云图能够从空间分布角度展示结果场变化情况，更适合用于说明程序已经具备从数值求解到图形化展示的完整后处理链路。因此，将这些图片放在验证算例部分更能直接对应“算例结果展示”的叙述逻辑。

从图 1 中预留的 7 个图位可以看出，本文已为后续的 ParaView 结果展示准备了完整的位置，其中前六张对应六个应力分量云图，最后一张对应位移模云图。待相应图片导出并保存到指定路径后，即可直接替换占位框并重新编译文档。这说明本文工作不仅实现了 H8 单元在 C++ 主程序中的读入、组装、求解和文本输出，而且已经具备将结果进一步转化为图形化后处理成果的能力，从而使算例验证更完整、结果表达更直观。

这一算例说明，H8 单元已经不再是程序枚举中的占位符，而是能够在 C++ 主程序中完整参与数据读入、矩阵组装、总体求解和结果输出。与此同时，原有 STAPpp 的天空线存储结构与 LDLT 求解器可以直接承载 H8 的 24 自由度单元，说明当前实现已经打通了 H8 单元在原框架中的最小可运行闭环。

在此手动插入 σ_{xx} 云图
可将上方注释中的
`\includegraphics` 路径改为你的图
片文件名

(a) σ_{xx} 云图

在此手动插入 σ_{yy} 云图
可将上方注释中的
`\includegraphics` 路径改为你的图
片文件名

(b) σ_{yy} 云图

在此手动插入 σ_{zz} 云图
可将上方注释中的
`\includegraphics` 路径改为你的图
片文件名

(c) σ_{zz} 云图

在此手动插入 τ_{xy} 云图
可将上方注释中的
`\includegraphics` 路径改为你的图
片文件名

(d) τ_{xy} 云图

在此手动插入 τ_{yz} 云图
可将上方注释中的
`\includegraphics` 路径改为你的图
片文件名

(e) τ_{yz} 云图

在此手动插入 τ_{xz} 云图
可将上方注释中的
`\includegraphics` 路径改为你的图
片文件名

(f) τ_{xz} 云图

在此手动插入位移模云图
可将上方注释中的
`\includegraphics` 路径改为你的图
片文件名

(g) 位移模云图

图 1: ParaView 后处理得到的 H8 验证算例云图预留位置：前六张为应力分量云图，最后一张为位移模云图

11 任务分工与合作情况

本次课程设计为个人作业，程序实现、算例验证、结果整理与报告撰写均由本人独立完成。

12 已完成功能与作业要求对应关系

对照课程作业要求，当前程序部分已经完成了 H8 单元在原始 STAPpp 主程序中的接入，实现了单元原理、数据输入、刚度组装、应力恢复与文本输出的基本闭环；同时完成了分片试验、收敛性分析和至少一个验证算例，并在 Python 原型中补充了 VTK 导出能力，可用于 ParaView 变形图和应力云图展示。若后续希望进一步提高程序完整性，则还可以继续将 VTK 导出直接集成到 C++ 主程序中，增加高斯点逐点应力输出、等效应力或主应力等更丰富的后处理量。

13 结论

本文完成了 8 节点六面体实体单元 H8 在 STAPpp 程序中的扩展，实现了从单元理论、材料定义、数据读入、刚度组装、应力恢复到主程序求解输出的完整闭环。同时，借助 Python 原型补充了分片试验、收敛性分析和 ParaView 后处理支持，使程序验证与结果展示更加完整。

综合各项结果可以认为，当前实现的 H8 单元已经通过畸变网格下的 patch test，在制造解驱动下表现出合理的收敛趋势，并能够在 C++ 主程序中完成单 H8 算例的完整求解与位移、应力输出。因此，本文工作已经满足“在 STAPpp 基础上新增 H8 单元并完成程序设计、验证和基本后处理”的核心要求，可作为课程报告的完整程序部分提交版本。